LD4 Automatic Density Backend Selection Specification

Transactional dense-versus-local-sparse planning for atomic and framework density fields

mdstats development specification

2026-07-20

Contents

1 Supersession note

This document records the LD4 implementation as introduced in `mdstats 0.19.50a0`. Its statement that dense storage is the production default is historical. LD11 in `mdstats 0.19.65a0` changes the shared defaults to the canonical periodized operator and `grid_backend="auto"` while preserving the LD4 candidate and selection algorithm. See `density_default_auto_policy_ld11_spec.md`.

2 Status and scope

This specification governs architecture gate **LD4** for `mdstats`. It begins from `mdstats 0.19.49a0`, where explicit dense and local block-sparse density preparation is available for atomic occupancy, framework-vertex occupancy, and framework-edge arc length.

LD4 enables

```
DensityStorageOptions(grid_backend="auto")
```

for those three channel types. Automatic selection is completed transactionally, before any floating scalar field is allocated. The selector compares exact dense and local-sparse Phase-B plans at the same scientific resolution, chooses one backend per field, verifies the complete scene against the Phase-C resource limits, and records the decision and both candidate estimates.

LD4 does **not** change the density estimator, Gaussian operator, broadening metric, edge source, edge quadrature, grid resolution, rendering method, or physical normalization. It does not implement performance caching, compiled kernels, parallel accumulation, or multilevel adaptive mesh refinement.

3 Scientific invariants

Automatic backend selection changes storage and evaluation strategy only. For each field, the following inputs are identical for the dense and sparse candidates:

- registered weighted samples;
- logical grid shape (N_1, N_2, N_3);
- display cell H;
- Gaussian bandwidth σ;
- canonical kernel-tail tolerance;
- broadening metric and adaptive-resolution result;
- framework edge source and resolved quadrature spacing;
- target total measure and physical units.

Auto mode must not increase the grid interval, reduce the grid shape, increase σ, loosen the kernel-tail tolerance, coarsen edge quadrature, drop samples, or change a requested HDR fraction to satisfy a resource limit.

The automatic backend requires `discrete_periodized_v1`. The legacy spectral operator remains dense-only because no scientifically identical local-sparse implementation exists for that versioned operator.

4 Public options

The existing option record becomes operational:

```
DensityStorageOptions(
    grid_backend="auto",                # dense | local_sparse | auto
    local_block_shape=(16, 16, 16),
    sparse_activation_fraction=0.20,
)
```

Constraints:

- `grid_backend` must be `dense`, `local_sparse`, or `auto`;
- `sparse_activation_fraction` lies in $(0, 1)$;
- `auto` requires `smoothing_operator="discrete_periodized_v1"`;
- explicit `dense` and explicit `local_sparse` retain their existing behavior;
- dense remains the default.

5 Candidate records

5.1 Candidate estimate

```
@dataclass(frozen=True)
class DensityBackendCandidateEstimate:
    backend: str
    feasible: bool
    logical_node_count: int
    active_node_count: int
    stored_value_count: int
    stored_block_count: int
    kernel_pair_count: int
    planning_bytes: int
    retained_bytes: int
    estimated_peak_bytes: int
    estimated_work: int
    infeasible_reason: str | None
    metadata: FrozenJSONMapping
```

The active and stored fractions are derived as

$$f_{\text{active}} = \frac{N_{\text{active}}}{N_{\text{logical}}}, \qquad f_{\text{stored}} = \frac{N_{\text{stored}}}{N_{\text{logical}}}.$$

Dense candidates use $N_{\text{active}} = N_{\text{stored}} = N_{\text{logical}}$. Sparse candidates use the exact target-node set and exact fixed-block packing plan.

5.2 Selection record

```
@dataclass(frozen=True)
class DensityBackendSelection:
    field_key: str
    requested_backend: str
    selected_backend: str
    reason: str
    dense: DensityBackendCandidateEstimate
    local_sparse: DensityBackendCandidateEstimate
    policy: str
    globally_overridden: bool
    metadata: FrozenJSONMapping
```

Candidate and selection records use canonical JSON serialization and exact schema versions. Every realized automatic field stores the complete selection record in its scientific metadata.

6 Exact Phase-B candidates

6.1 Dense candidate

The dense candidate uses the current exact dense Phase-B plan. Its hard feasibility checks include logical voxels, samples, sample bytes, planning bytes, stencil values, component values, mesh cells, mesh faces, rendered points, and estimated package-owned peak memory.

A deterministic work proxy is

$$W_{\text{dense}} = 8N_s + N_g \max\bigl(1, \lceil \log_2 \max(2, N_g) \rceil\bigr) + N_g,$$

where N_s is the sample count and $N_g = N_1 N_2 N_3$ is the logical node count. The proxy ranks alternatives; it is not a wall-time prediction.

6.2 Local-sparse candidate

The sparse candidate performs the exact bounded planning operations already required by LD1-B:

1. aggregate periodic CIC contributions into sorted occupied logical nodes;
2. enumerate the canonical finite kernel support;
3. determine the exact target-node union;
4. pack target nodes into the requested periodic fixed blocks;
5. count kernel pairs, stored block slots, masks, planning arrays, and retained bytes.

Its work proxy is

$$W_{\text{sparse}} = 8N_s + N_{\text{pairs}} + N_{\text{stored}}.$$

Failure of one candidate does not fail auto mode if the other candidate is feasible. If neither candidate is feasible, preparation fails before scalar allocation and reports both reasons.

7 Field-local policy

For two feasible candidates, the normative anchors are:

```
sparse active fraction >= 0.50
    -> dense: broad_active_fraction

sparse active fraction <= sparse_activation_fraction
and sparse peak bytes <= 0.70 * dense peak bytes
    -> local_sparse: localized_and_memory_efficient

otherwise
    -> lower estimated peak bytes
    -> then lower estimated work
    -> then dense on an exact tie
```

If only dense is feasible, select dense with reason `sparse_infeasible`. If only sparse is feasible, select sparse with reason `dense_infeasible`.

These rules are project-specific architecture policy. They are not borrowed from a published automatic-mesh or database-selection algorithm.

8 Whole-scene transactional selection

A scene may contain up to the existing `max_density_fields` limit. For each field, the planner retains all feasible candidates. It then deterministically enumerates the Cartesian product of available backend choices and applies the existing Phase-C scene approval to each combination.

The score is ordered lexicographically by

$$\left(N_{\text{policy overrides}}, B_{\text{scene peak}}, \sum_f W_f, N_{\text{sparse}} \right).$$

Thus the field-local policy is preserved whenever the complete scene is feasible. A field is globally overridden only when required by scene-wide limits. The selected record then carries

```
globally_overridden = true
reason = global_resource_override_from_<preferred backend>
```

The existing maximum of eight density fields bounds exhaustive enumeration by $2^8 = 256$ combinations. Selection order and output are independent of hash iteration and thread scheduling.

9 Standalone preparation

`prepare_atomic_density_fields()` and `prepare_framework_density_fields()` may be called outside a composite framework-dynamics scene. In that case, each field performs the same exact dense and sparse preflight and applies the field-local policy. No scene-wide override is possible because no other field is in scope.

10 Realization and metadata

After Phase-C approval, each field is constructed using the selected backend. Its metadata records at least:

```
requested_storage_backend
storage_backend
backend_selection.schema_version
backend_selection.selected_backend
backend_selection.reason
backend_selection.globally_overridden
both candidate feasibility states
both candidate active/stored fractions
both candidate peak/work estimates
policy anchors
scene approval identifier, when applicable
```

The realized field must agree with its selected Phase-B plan under the existing realization checks. Renderer dispatch remains capability-based and is unchanged.

11 Resource and failure policy

Auto selection completes before allocating dense scalar arrays or sparse block-value arrays. Exact integer planning arrays remain bounded by the LD0-R3 planning limits.

The selector must:

- never silently fall back after a realization failure;
- never try dense first and allocate it speculatively;
- never catch scientific validation errors as resource alternatives;
- preserve the exact requested resolution when dense exceeds `max_density_voxels`;
- choose sparse only when its own hard limits pass;
- fail with both candidate reasons when neither backend is feasible.

12 Acceptance gates

12.1 Anchored outcomes

- A broad field with $f_{\text{active}} \geq 0.50$ selects dense.
- A localized field satisfying both sparse anchors selects local sparse.
- Intermediate fields use the documented peak/work/tie order.
- Changing dictionary or candidate insertion order does not change the result.

12.2 Scientific equivalence

For any selected backend, the auto field must be identical to the corresponding explicitly forced backend at the same options:

```
relative L1 field difference        = 0 within floating roundoff
relative Linfinity field difference = 0 within floating roundoff
integral difference                 <= 5e-13 * max(1, total_measure)
HDR threshold difference            <= 5e-12 relative
```

The logical grid shape, resolved intervals, Gaussian bandwidth, kernel support, broadening diagnostics, and edge-quadrature policy must be exactly equal.

12.3 Transactional behavior

- Selection is present in the approved Phase-B record before the first field constructor is called.
- A global resource conflict produces a deterministic feasible override when one exists.
- No feasible combination produces `GraphComplexityError` before scalar allocation.
- Selection records round-trip through canonical JSON.

12.4 Compatibility

- Explicit dense output remains byte-for-byte compatible with `mdstats 0.19.49a0`.
- Explicit local-sparse output remains numerically and topologically unchanged.
- `auto` with `legacy_spectral_v1` is rejected.
- Dense remains the default backend.

13 Required focused tests

1. broad active-fraction dense anchor;
2. localized active-fraction and peak-ratio sparse anchor;
3. intermediate peak-memory choice;
4. work-proxy tie break and final dense tie;
5. dense-infeasible and sparse-infeasible one-candidate cases;
6. neither-candidate failure with both reasons;
7. exact auto-versus-forced atomic equivalence;
8. exact auto-versus-forced framework-vertex equivalence;
9. exact auto-versus-forced framework-edge equivalence;
10. preservation of a logical grid exceeding the dense voxel budget;
11. global scene override under aggregate resource limits;
12. selection determinism and canonical JSON round trip;
13. exact realized-versus-planned counts;
14. dense and explicit-sparse regression suites;
15. wheel, source distribution, and source-ZIP smoke tests.

14 Completion condition

LD4 is complete when automatic selection is operational for all three production density channel types, every decision occurs before scalar allocation, broad and localized anchors are certified, whole-scene limits can override field-local choices deterministically, and explicit dense and explicit sparse behavior remains unchanged.